实验报告01：Python基础与数据处理入门

一、实验环境与提交要求

1. 请使用独立的conda虚拟环境完成实验，不得使用base环境。
2. 使用Python 3和Jupyter Notebook；可在PyCharm或VS Code中完成实验。
3. 请先将第一个代码框中的学号修改为自己的学号，再完成以下实验，并按顺序运行各个代码框。
4. 使用Markdown单元格解释思路（有必要的话），使用代码单元格完成计算。
5. 提交前重启内核并运行全部代码，检查是否存在依赖旧变量、遗漏文件或路径错误的问题。
6. 将Notebook命名为`学号_姓名_exp01.ipynb`。

1.1 第一个代码框：身份与环境记录

```
import os, uuid, hashlib, platform, getpass

# ===== 1. 填写学号 =====
STUDENT_ID = 20190010  #修改为自己的学号

# ===== 2. 打印 conda 环境 =====
CONDA_ENV = os.environ.get("CONDA_DEFAULT_ENV", "base")
print(f"[Conda 环境] {CONDA_ENV}")

# ===== 3. 机器指纹（绑定当前电脑）=====
MACHINE_FP = hashlib.sha256(
    f"{platform.node()}|{platform.machine()}|{platform.system()}|
{uuid.getnode()}".encode()
).hexdigest()[:16]

# ===== 4. 生成唯一 ID =====
RUN_ID = f"{STUDENT_ID}-{MACHINE_FP}"
print(f"[学号]     {STUDENT_ID}")
print(f"[机器指纹] {MACHINE_FP}")
print(f"[RUN_ID]   {RUN_ID}")
```

二、实验内容与过程

实验1：基本运算、数据类型与字符串

操作任务：

1. 分别计算`8 / 3`、`8 // 3`、`8 % 3`和`2 ** 7`，输出结果及其类型，说明各运算符的含义。
2. 创建整数、浮点数、字符串和布尔值各一个，使用`type()`检查其类型。
3. 执行`int("123")`、`float("3.14")`和`int(3.9)`，解释转换结果；捕获`int("goat")`引发的异常并说明原因。
4. 对字符串`" Python Data Analysis "`完成去除首尾空白、大小写转换、切片及单词拆分，再使用`join()`将单词连接起来。
5. 比较字符串拼接与重复操作；验证字符串不支持直接修改单个字符，并给出构造新字符串的方式。

实验2：函数、条件、循环与递归

操作任务：

1. 编写`score_level(score)`函数：90～100分返回“优秀”，80～不足90分返回“良好”，60～不足80分返回“及格”，0～不足60分返回“不及格”。对于超出0～100范围的数值，抛出`ValueError`。
2. 分别使用`for`和`while`计算1到100的整数和，比较两种实现的结果。
3. 编写递归版和迭代版阶乘函数，约定`0! = 1`，输入为非负整数。比较`n = 0、1、5、10`时的结果。
4. 编写`summary(values)`函数，接收非空数值列表，以元组返回最小值、最大值和平均值，并通过元组解包接收结果。
5. 编写`sum_all(*args)`函数，返回所有参数之和；分别测试无参数、单个参数和多个参数的情况。

实验3：列表、字典、元组与基础统计

使用以下模拟成绩数据，不需要输入真实学生信息：

```
names = ["Alice", "Bob", "Carol", "David", "Eva", "Frank"]
scores = [85, 58, 92, 76, 60, 89]
```

操作任务：

1. 使用`zip()`构建姓名到成绩的字典，使用`items()`逐项输出姓名和成绩。
2. 使用`enumerate()`输出序号、姓名和成绩，序号从1开始。
3. 使用列表推导式提取及格学生的姓名，并生成所有成绩的平方列表。
4. 计算人数、平均分、最高分、最低分和及格率，使用字符串格式化将平均分显示为两位小数、及格率显示为百分数。
5. 比较`alias = scores`与`copied = scores.copy()`：修改`alias`中的一个元素，观察原列表和副本的变化。该观察任务使用单独的演示列表，避免改变后续实验的原始成绩。
6. 使用元组保存一条学生记录，验证其元素不能直接重新赋值；观察长度不同的两个序列使用`zip()`时的结果。

实验4：文件读写、异常处理与模块

在Notebook同级目录创建`exp01_data`文件夹，保存以下UTF-8文本文件`students.txt`。每行字段用英文逗号分隔，本实验不包含字段内逗号或引号等复杂情况。

```
name,score
Alice,85
Bob,58
Carol,92
David,76
Eva,60
Frank,89
Grace,abc
Henry,105
Incomplete
```

操作任务：

1. 使用`os.getcwd()`查看工作目录，使用`os.path.join()`构造相对路径，并使用`os.makedirs(..., exist_ok=True)`创建数据目录。
2. 使用`with open(..., encoding="utf-8")`写入上述数据，再读取文件，跳过表头，使用字符串方法解析记录。
3. 编写`parse_student(line)`函数：将成绩转换为整数，检查字段数量、姓名是否为空及成绩是否在0～100范围内。无效记录抛出`ValueError`。
4. 按行处理文件，捕获无效记录的异常，记录行号与错误原因，并继续处理后续记录。有效记录保存为字典组成的列表。
5. 将有效记录数、无效记录数、平均分和及格率写入`exp01_data/report.txt`。分别演示`w`与`a`模式，说明二者对已有内容的影响。

6. 将有效记录使用`pickle.dump()`保存为`exp01_data/students.pkl`，使用`pickle.load()`读取自己生成的文件，验证恢复的数据与原数据相等。
7. 将`parse_student()`保存到Notebook同级的`student_utils.py`模块中，通过`import`调用，并展示一个有效输入和一个无效输入的测试结果。
8. 读取一个明确不存在的文件，捕获`FileNotFoundError`并输出说明。

实验5：类、继承与对象复制

操作任务：

1. 定义`Student`类，在`__init__()`中初始化`name`和`score`属性，并实现`is_passed()`和`__str__()`方法。
2. 为实验4中的每条有效记录创建一个`Student`对象，输出对象信息和及格状态。
3. 定义继承自`Student`的`GraduateStudent`类，增加`research_topic`属性，通过`super().__init__()`初始化父类属性，并重写`__str__()`。
4. 构造包含嵌套列表的对象，例如`{"name": "Alice", "scores": [85, 90]}`。分别进行直接赋值、`copy.copy()`和`copy.deepcopy()`，修改内部列表，比较结果。每次比较都从新建的原始对象开始。